



INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIA E CIÊNCIAS
DEPARTAMENTO DE TECNOLOGIA E CIÊNCIAS
ENGENHARIA INFORMÁTICA

PROJECTO FINAL DE ESTRUTURA DE DADOS II

MPEGA

Sistema Inteligente de Partilha de Viagens

INTEGRANTES DO GRUPO

Briana Vicente– 20232111
Delfina Ramos– 20230956
Rodolfo Guzman - 20230090

CURSO: Engenharia Informática

TURMA: EINF4 - T3

DATA: 06/07/2026

DOCENTE: Sílvia António

GRUPO: 13

Índice

1	INTRODUÇÃO	3
2	OBJECTIVOS	1
2.1	Objectivo Geral	1
2.2	Objectivos Específicos	1
3	PROBLEMÁTICA	2
4	JUSTIFICATIVA	2
5	METODOLOGIA	3
5.1	Análise do Problema	3
5.2	Definição das Classes	3
5.3	Planeamento e Implementação	4
5.4	Testes e Validação	4
6	CLASSES PRINCIPAIS	4
6.1	Módulo Hash	4
6.2	Módulo Grafo	5
6.3	Módulo Viagem	6
6.4	Módulo Ficheiros	6
6.5	Módulo Dashboard	7
7	GESTÃO DE FICHEIROS	7
8	FUNCIONALIDADES E INTERFACE	8
9	BIBLIOTECAS UTILIZADAS	9
10	DIVISÃO DE TAREFAS	9
11	CONCLUSÃO	11
12	REPOSITÓRIO GITHUB	11
	REFERÊNCIAS	12

1 INTRODUÇÃO

O presente relatório descreve o desenvolvimento do **MPEGA – Sistema Inteligente de Partilha de Viagens**, uma aplicação de consola desenvolvida na linguagem **C**, no âmbito da disciplina de **Estruturas de Dados II**. O projeto teve como objetivo aplicar, de forma prática, os conhecimentos adquiridos ao longo da disciplina, nomeadamente a utilização de Tipos Abstratos de Dados (TAD), tabelas hash, listas ligadas, grafos e manipulação de ficheiros.

O sistema foi desenvolvido para facilitar a gestão e a partilha de viagens entre condutores e passageiros, permitindo o registo e autenticação de utilizadores com diferentes níveis de acesso (Administrador, Conductor e Passageiro). Além disso, possibilita a gestão de cidades e rotas, a publicação e reserva de viagens e a pesquisa da melhor rota através do algoritmo de Dijkstra.

Toda a informação é armazenada em ficheiros de texto, garantindo a persistência dos dados entre diferentes execuções do programa. Desta forma, o projeto demonstra a aplicação integrada de diferentes estruturas de dados na construção de uma aplicação organizada, eficiente e funcional.

2 OBJECTIVOS

2.1 Objectivo Geral

Desenvolver um sistema inteligente de partilha de viagens em linguagem **C** que permita a gestão de utilizadores, cidades, rotas e viagens, utilizando estruturas de dados como Tabelas Hash, Listas Ligadas e Grafos, garantindo uma gestão eficiente da informação e a persistência dos dados através de ficheiros.

2.2 Objectivos Específicos

- Desenvolver um sistema de autenticação para Administrador, Conductor e Passageiro.
- Implementar a gestão de utilizadores utilizando a Tabela Hash.
- Permitir o registo, consulta e gestão de cidades e rotas através de Grafos.
- Implementar a publicação, reserva, início e término de viagens.

- Determinar a melhor rota entre duas cidades utilizando o algoritmo de Dijkstra.
- Armazenar e recuperar os dados do sistema através de ficheiros de texto, garantindo a persistência da informação.
- Desenvolver uma interface de consola simples, organizada e de fácil utilização.

3 PROBLEMÁTICA

No contexto angolano, a utilização do transporte informal para deslocações diárias constitui uma realidade para grande parte da população. No entanto, o processo de conseguir um táxi ou uma boleia é frequentemente desorganizado, exigindo que os passageiros disputem um lugar disponível e, muitas vezes, sem saber qual o veículo que está a parar, qual será o seu destino ou se ainda possui vagas. Além disso, existem condutores que oferecem boleias gratuitas ou pagas, mas muitos passageiros acabam por perder essa oportunidade por falta de informação ou por outras pessoas chegarem primeiro.

Esta realidade evidencia a necessidade de uma solução tecnológica que facilite a comunicação entre condutores e passageiros, permitindo a publicação e pesquisa de viagens de forma organizada. Assim, o desenvolvimento do sistema **MPEGA – Sistema Inteligente de Partilha de Viagens** procura minimizar estas dificuldades, oferecendo uma plataforma onde os condutores podem divulgar as suas viagens e os passageiros podem procurar e reservar lugares de acordo com as suas necessidades, tornando o processo mais eficiente, organizado e seguro.

4 JUSTIFICATIVA

O desenvolvimento do sistema **MPEGA – Sistema Inteligente de Partilha de Viagens** justifica-se pela necessidade de melhorar a organização do transporte partilhado em Angola, tornando mais simples a comunicação entre condutores e passageiros. Além de responder a um problema do quotidiano, o projeto permitiu aplicar, na prática, os conhecimentos adquiridos na disciplina de Estruturas de Dados II, através da utilização de Tabelas Hash, Listas Ligadas, Grafos e manipulação de ficheiros, proporcionando uma solução funcional, organizada e de fácil utilização.

5 METODOLOGIA

O desenvolvimento do sistema foi organizado em etapas bem definidas, de forma a garantir uma implementação estruturada e eficiente. Inicialmente, foram levantados os requisitos do sistema e definidas as funcionalidades destinadas ao Administrador, Condutor e Passageiro. Posteriormente, procedeu-se à implementação da aplicação em linguagem C, recorrendo à programação modular, com o código distribuído por diferentes ficheiros, nomeadamente **hash** (gestão de utilizadores), **grafo** (gestão de cidades e rotas), **viagem** (gestão das viagens e reservas), **ficheiros** (persistência dos dados) e **dashboard** (interface e menus do sistema). Para a implementação foram utilizadas **Tabelas Hash**, **Grafos**, **Listas Ligadas** e **ficheiros de texto**, permitindo uma gestão eficiente da informação e a persistência dos dados entre diferentes execuções do programa.

5.1 Análise do Problema

A primeira etapa do desenvolvimento consistiu na análise dos requisitos do sistema e na identificação das principais necessidades do problema. Foram estudadas as funcionalidades essenciais para a gestão da partilha de viagens, incluindo o registo e autenticação de utilizadores, a gestão de cidades e rotas, a publicação de viagens pelos condutores e a pesquisa e reserva de viagens pelos passageiros. Esta análise permitiu definir a estrutura do sistema, as relações entre os diferentes módulos e as estruturas de dados mais adequadas, nomeadamente Tabelas Hash para a gestão dos utilizadores, Grafos para a representação das cidades e rotas, Listas Ligadas para a gestão das reservas e ficheiros de texto para garantir a persistência dos dados.

5.2 Definição das Estruturas de dados

Após a análise dos requisitos, foram definidas as principais estruturas de dados responsáveis por representar as entidades do sistema. Para a modelação dos dados foram utilizadas estruturas (**struct**) em linguagem C, representando utilizadores, cidades, rotas, viagens e reservas. O sistema foi organizado de forma modular, distribuindo as funcionalidades por diferentes ficheiros, como **hash**, responsável pela gestão dos utilizadores; **grafo**, para a representação das cidades e rotas; **viagem**, para a gestão das viagens e reservas; **ficheiros**, para a persistência dos dados; e **dashboard**, responsável pela interface e interação com o utilizador. Esta organização permitiu uma melhor separação das responsabilidades, facilitando o desenvolvimento, manutenção e evolução da aplicação.

5.3 Planeamento e Implementação

Após a análise dos requisitos, o sistema foi implementado de forma modular, desenvolvendo inicialmente a gestão de utilizadores, seguida da gestão de cidades, rotas e viagens. Por fim, foi implementada a persistência dos dados em ficheiros de texto e realizados testes para garantir o correto funcionamento de todas as funcionalidades.

5.4 Testes e Validação

Após a implementação, foram realizados testes funcionais em todos os módulos do sistema, verificando o registo e autenticação de utilizadores, a gestão de cidades, rotas e viagens, bem como a pesquisa, reserva e cancelamento de viagens. Também foram testadas a persistência dos dados em ficheiros e a correta interação entre os diferentes perfis de utilizadores, confirmando o funcionamento adequado da aplicação.

6 PRINCIPAIS MÓDULOS DO SISTEMA

O sistema foi desenvolvido de forma modular, distribuindo as suas funcionalidades por diferentes ficheiros, cada um com uma responsabilidade específica. Esta organização permitiu uma melhor separação das tarefas, facilitando o desenvolvimento, a manutenção e a reutilização do código. Cada módulo é responsável por uma parte do sistema, como a gestão dos utilizadores, cidades, rotas, viagens, persistência dos dados e interface com o utilizador, contribuindo para uma aplicação mais organizada e eficiente.

6.1 Módulo Hash

O módulo **Hash** é responsável pela gestão dos utilizadores do sistema. Nele são implementadas as operações de registo, autenticação, pesquisa, atualização, bloqueio e remoção de utilizadores, utilizando uma **Tabela Hash** para proporcionar pesquisas rápidas e eficientes através do identificador de cada utilizador.

Tabela 1 — Principais estruturas e funções do módulo Hash

Estrutura/Função	Tipo	Descrição
Usuario	Estrutura	Representa um utilizador do sistema.
Lista	Estrutura	Representa cada posição da Tabela Hash.
inicializarTabela()	void	Inicializa a Tabela Hash.
inserir_usuario()	void	Insere um utilizador na Tabela Hash.
buscarUsuario()	No*	Procura um utilizador pelo identificador.

Estrutura/Função	Tipo	Descrição
atualizarUsuario()	int	Atualiza os dados de um utilizador.
removerUsuario()	int	Remove um utilizador da Tabela Hash.
bloquearDesbloquearUsuario())	int	Altera o estado do utilizador.
login()	Usuario*	Valida as credenciais de acesso.
guardarUsuarios()	void	Guarda os utilizadores em ficheiro.
carregarUsuarios()	void	Carrega os utilizadores do ficheiro.

Fonte: Elaboração própria (2026).

6.2 Módulo Grafo

O módulo **Grafo** é responsável pela gestão das cidades e das rotas do sistema. Através desta estrutura é possível registar cidades, criar ligações entre elas, atualizar o estado das vias e determinar o melhor percurso entre uma origem e um destino utilizando o algoritmo de Dijkstra.

Tabela 2 — Principais estruturas e funções do módulo Grafo

Estrutura/Função	Tipo	Descrição
Cidade	struct	Representa uma cidade do sistema.
Rota	struct	Representa uma ligação entre duas cidades.
inicializarGrafo()	String	Inicializa o grafo e a matriz de adjacência.
adicionarCidade()	void	Regista uma nova cidade.
removerCidade()	String	Remove uma cidade e as respetivas ligações.
adicionarRota()	int	Adiciona uma nova rota entre duas cidades.
removerRota()	boolean	Remove uma rota existente.
listarCidades()	void	Apresenta todas as cidades registadas.
listarRotas()	void	Apresenta todas as rotas registadas.
atualizarEstadoVia()	int	Atualiza o estado de uma rota.
procurarCidadePorNome()	Usuario	Procura uma cidade pelo nome.
dijkstra()	void	Determina o melhor percurso entre duas cidades.
guardarCidades()	void	Guarda as cidades em ficheiro.
carregarCidades()	void	Carrega as cidades do ficheiro.
guardarRotas()	void	Guarda as rotas em ficheiro.
carregarRotas()	void	Carrega as rotas do ficheiro.

Fonte: Elaboração própria (2026).

6.3 Módulo Viagem

O módulo **Viagem** é responsável pela gestão das viagens publicadas pelos condutores e pelas reservas efetuadas pelos passageiros. Neste módulo são implementadas as operações de publicação, pesquisa, reserva, cancelamento, início e término das viagens, bem como a gestão dos passageiros associados a cada viagem.

Tabela 3 — Principais estruturas e funções do módulo Viagem

Estrutura/Função	Tipo	Descrição
Viagem	struct	Representa uma viagem publicada por um condutor.
NoPassageiro	struct	Representa um passageiro associado a uma viagem.
publicarViagem()	int	Publica uma nova viagem.
listarViagens()	void	Lista todas as viagens registadas.
listarMinhasViagens()	void	Lista as viagens publicadas por um condutor.
procurarViagem()	void	Procura viagens por origem e destino.
buscarViagem()	Viagem*	Procura uma viagem pelo identificador.
reservarLugar()	int	Efetua a reserva de um lugar numa viagem.
cancelarReserva()	int	Cancela a reserva de um passageiro.
listarPassageiros()	void	Lista os passageiros de uma viagem.
iniciarViagem()	int	Altera o estado da viagem para "Em Curso".
terminarViagem()	int	Altera o estado da viagem para "Terminada".
removerViagem()	int	Remove uma viagem do sistema.

Fonte: Elaboração própria (2026).

6.4 Módulo Ficheiros

O módulo **Ficheiros** é responsável pela persistência dos dados do sistema. Este módulo permite guardar e recuperar as informações dos utilizadores, cidades, rotas, viagens e reservas através de ficheiros de texto, garantindo que os dados permaneçam disponíveis entre diferentes execuções da aplicação.

Estrutura/Função	Tipo	Descrição
guardarUsuarios()	void	Guarda os utilizadores em <code>usuarios.txt</code> .
carregarUsuarios()	void	Carrega os utilizadores do ficheiro.
guardarCidades()	void	Guarda as cidades em <code>cidades.txt</code> .
carregarCidades()	void	Carrega as cidades do ficheiro
guardarRotas()	void	Guarda as rotas em <code>rotas.txt</code> .

Estrutura/Função	Tipo	Descrição
carregarRotas()	void	Carrega as rotas do ficheiro.
guardarViagens()	void	Guarda as viagens em viagens.txt .
carregarViagens()	void	Carrega as viagens do ficheiro.
guardarReservas()	void	Guarda as reservas em reservas.txt .
carregarReservas()	void	Carrega as reservas do ficheiro.

Fonte: Elaboração própria (2026).

6.5 Módulo Dashboard

O módulo **Dashboard** é responsável pela interação entre o utilizador e o sistema. Neste módulo encontram-se implementados os menus principais e específicos de cada perfil de utilizador (Administrador, Condutor e Passageiro), bem como as funcionalidades de autenticação, navegação e apresentação das informações. É através deste módulo que o utilizador acede às diferentes operações disponibilizadas pelo sistema de forma organizada e intuitiva.

Estrutura/Função	Tipo	Descrição
menuAdmin()	int	Apresenta o menu do Administrador.
menuCondutor()	int	Apresenta o menu do Condutor.
menuPassageiro()	int	Apresenta o menu do Passageiro

7 GESTÃO DE FICHEIROS

A gestão de ficheiros foi implementada com o objetivo de garantir a persistência dos dados do sistema entre diferentes execuções. Para isso, foram desenvolvidas funções responsáveis por guardar e carregar as informações dos utilizadores, cidades, rotas, viagens e reservas em ficheiros de texto. Desta forma, os dados registados durante a utilização do sistema permanecem armazenados, evitando a perda de informação quando a aplicação é encerrada e permitindo a sua recuperação na próxima execução.

Tabela 4 — Estrutura dos ficheiros

Ficheiro	Formato de cada registo
usuarios.txt	id;nome;telefone;email;senha;tipo;estado
idades.txt	idCidade;nomeCidade
rotas.txt	idRota;origem;destino;distancia;tempoEstimado;estadoVia
viagens.txt	idViagem;idCondutor;idVeiculo;origem;destino;distancia;tempoEstimad
reservas.txt	idViagem;idPassageiro

Fonte: Elaboração própria (2026).

8 FUNCIONALIDADES E INTERFACE

A interação com o sistema é realizada através de menus textuais apresentados de forma simples e organizada. Inicialmente, o utilizador pode criar uma conta ou iniciar sessão através do seu correio eletrónico e palavra-passe. Caso não exista nenhum utilizador registado, o primeiro utilizador criado é automaticamente definido como Administrador.

Após a autenticação, o sistema apresenta um menu específico de acordo com o perfil do utilizador. O **Administrador** é responsável pela gestão dos utilizadores, cidades e rotas; o **Condutor** pode publicar, gerir e acompanhar as suas viagens, bem como consultar a melhor rota para o destino pretendido; e o **Passageiro** pode pesquisar viagens disponíveis, efetuar reservas e cancelar reservas.

As principais funcionalidades do sistema são:

- **Gestão de Utilizadores:** registar, autenticar, atualizar, bloquear/desbloquear, remover e listar utilizadores.
- **Gestão de Cidades e Rotas:** registar, listar, atualizar e remover cidades e rotas, bem como consultar o melhor percurso entre duas cidades através do algoritmo de Dijkstra.
- **Gestão de Viagens:** publicar, pesquisar, listar, iniciar, terminar e remover viagens, além de gerir as reservas dos passageiros.
- **Persistência dos Dados:** guardar e carregar utilizadores, cidades, rotas, viagens e reservas através de ficheiros de texto.

Durante a utilização do sistema, são apresentadas mensagens de confirmação e de erro, permitindo orientar o utilizador sempre que uma operação é concluída com sucesso ou quando ocorre alguma situação que impeça a sua execução.

9 BIBLIOTECAS UTILIZADAS

No desenvolvimento do sistema foram utilizadas exclusivamente bibliotecas da linguagem padrão C, sem recurso a bibliotecas externas. As principais bibliotecas utilizadas são as seguintes:

- **stdio.h:** utilizada para operações de entrada e saída de dados, como a leitura das informações introduzidas pelo utilizador (`scanf`) e a apresentação de mensagens no ecrã (`printf`), bem como para a leitura e escrita dos ficheiros de texto (`FILE`, `fopen`, `fprintf`, `fscanf` e `fclose`).
- **stdlib.h:** utilizada para funções auxiliares do sistema, nomeadamente a gestão dinâmica de memória (`malloc` e `free`) na implementação das listas ligadas, bem como para funções de controlo da aplicação, como a limpeza do ecrã através da função `system()`.
- **string.h:** utilizada na manipulação de cadeias de caracteres, permitindo copiar, comparar e processar informações como nomes, correios eletrónicos, cidades, estados das vias e palavras-passe, através de funções como `strcpy()` e `strcmp()`.

10 DIVISÃO DE TAREFAS

Apesar da divisão das responsabilidades, todas as etapas foram discutidas e ajustadas em conjunto. A colaboração e a troca de ideias foram centrais para o sucesso do projeto. Em seguida, descreve-se como as responsabilidades foram divididas:

Tabela 5 — Distribuição de responsabilidades do grupo

Membro	Responsabilidades
Briana Vincente	Responsável pelo desenvolvimento do módulo de gestão de utilizadores, implementando a Tabela Hash, o cadastro, autenticação, pesquisa, atualização, bloqueio/desbloqueio e remoção de utilizadores, bem como as estruturas de dados relacionadas com os utilizadores.
Delfina Ramos	Responsável pelo desenvolvimento do módulo Grafo, implementando a gestão de cidades e rotas, inserção e remoção de cidades e rotas, atualização do estado das vias e o algoritmo de Dijkstra para determinação do melhor percurso entre duas cidades.
Rodolfo Guzman	Responsável pela integração (merge) dos módulos desenvolvidos pelo grupo, desenvolvimento da interface e dos menus do sistema, implementação e melhoria da persistência dos dados em ficheiros, desenvolvimento do módulo de viagens e reservas, integração das funcionalidades dos perfis de Administrador,

Membro	Responsabilidades
	Condutor e Passageiro, bem como pela elaboração do relatório do projeto.

Fonte: Elaboração própria (2026).

A concepção do sistema foi pensada em conjunto por todos os membros. Durante o desenvolvimento, foram simulados cenários reais de utilização de uma biblioteca para validar decisões e ajustar funcionalidades. As reuniões foram marcadas por trocas de ideias, sugestões e revisões em grupo, permitindo que cada membro contribuísse activamente. Essa colaboração fortaleceu o trabalho em equipa e proporcionou crescimento técnico e interpessoal.

11 CONCLUSÃO

O desenvolvimento do **MPEGA – Sistema Inteligente de Partilha de Viagens** permitiu aplicar, de forma prática, os conhecimentos adquiridos na disciplina de **Estruturas de Dados II**, recorrendo à utilização de Tabelas Hash, Grafos, Listas Ligadas e manipulação de ficheiros para a construção de uma aplicação funcional e organizada.

Os objetivos propostos foram alcançados, sendo possível desenvolver um sistema capaz de gerir utilizadores, cidades, rotas, viagens e reservas, garantindo diferentes níveis de acesso para Administradores, Condutores e Passageiros, bem como a persistência dos dados entre diferentes execuções da aplicação.

O maior desafio enfrentado durante o desenvolvimento do projeto foi integrar diferentes estruturas de dados, algoritmos e abordagens de implementação num único sistema funcional. A junção de módulos desenvolvidos separadamente, como a Tabela Hash, o Grafo, a gestão de viagens e o sistema de persistência em ficheiros, exigiu um trabalho contínuo de adaptação, integração e testes para garantir o correto funcionamento da aplicação.

Além do reforço dos conhecimentos técnicos, o projeto contribuiu para o desenvolvimento da capacidade de trabalho em equipa, da organização modular do código e da resolução de problemas durante a implementação. Embora existam funcionalidades que poderão ser aperfeiçoadas em versões futuras, o sistema desenvolvido cumpre os objetivos inicialmente definidos, constituindo uma solução funcional para apoiar a gestão da partilha de viagens.

12 REPOSITÓRIO GITHUB

O código-fonte do projecto encontra-se publicado no repositório GitHub do grupo, onde estão disponíveis todas as classes e a estrutura final do projecto. O endereço do repositório é o seguinte:

[Link: <https://github.com/Bellbelliones/MPEGA>]

REFERÊNCIAS

Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.

Deitel, P. J., & Deitel, H. M. (2016). *C: How to Program* (8th ed.). Pearson.

Schildt, H. (2000). *C: The Complete Reference* (4th ed.). McGraw-Hill.

King, K. N. (2008). *C Programming: A Modern Approach* (2nd ed.). W. W. Norton & Company.

Harbison, S. P., & Steele Jr., G. L. (2002). *C: A Reference Manual* (5th ed.). Prentice Hall.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C* (2nd ed.). Pearson.